

Contents

- [Problem 1](#)
- [Problem 4](#)
- [Problem 6](#)
- [Problem 9](#)

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% ECE 341 Lab 2 - Time Domain Analysis of Continuous-Time Systems %
% Author: Gianni Avilan & Alyx Wilhelm                               %
% 02.16.2026                                                         %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

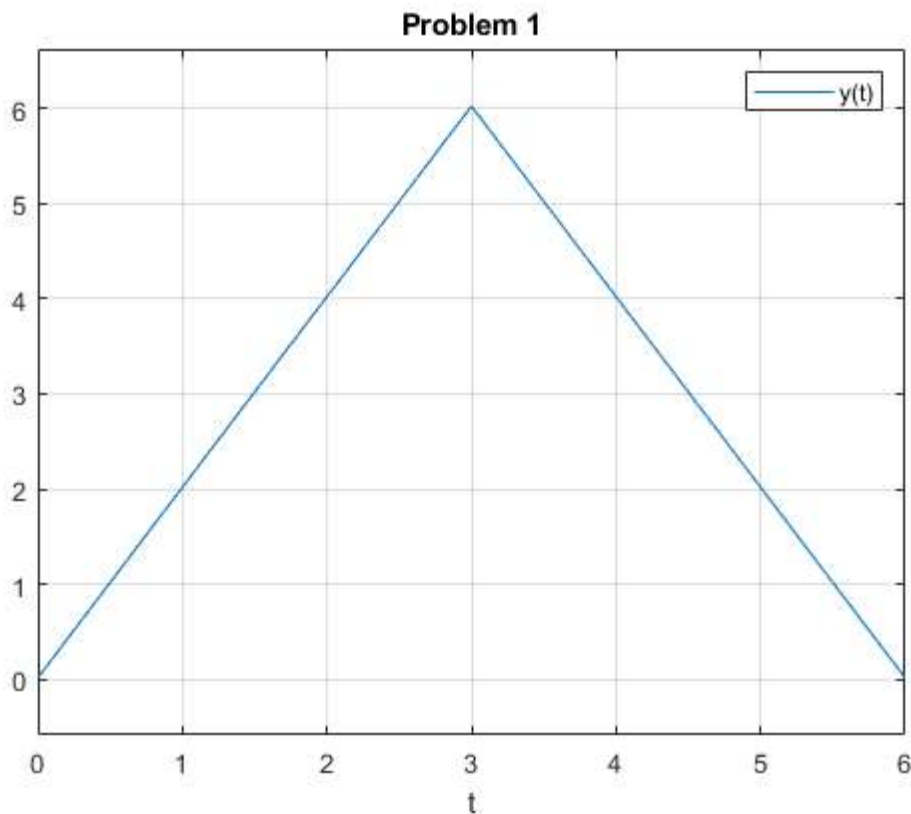
clear          % clear existing variables from the workspace
close all      % close all figures
clc            % clear the command window of text
```

Problem 1

```
clear;
close all;
clc;

%{
The impulse response  $h(t) = u(t) - u(t-3)$  is 0 everywhere other than
at  $0 \leq t \leq 3$ , and can be simplified to  $h(t) = 1$  for  $t \in (0,3)$ , and
 $h(t) = 0$  elsewhere. The input signal  $x(t)$ , on the other hand, is defined
as  $x(t) = 2$  for  $t \in (0,3)$ , and 0 elsewhere.
%}

%taking limits at the points where both functions are non-zero, a=0, b=3
%using a small step size, compute the time interval
a=0;b=3;
step = .01; t=a:step:b;
%compute x and h in this time interval
x=ones(size(t))*2; h = ones(size(t));
%{
convolve x and h (note that since conv() computes the entire overlap
between the two functions, which is over the interval (a,2b).
%}
y=conv(x,h)*step;
%set up a new time vector for the output signal y(t).
ty=a:step:2*b;
%plot the output
plot(ty,y); xlim([a,2*b]); ylim([min(y)-range(y)*0.1, max(y)+range(y)*0.1]);
xlabel('t'); legend('y(t)'); grid on; title('Problem 1');
```

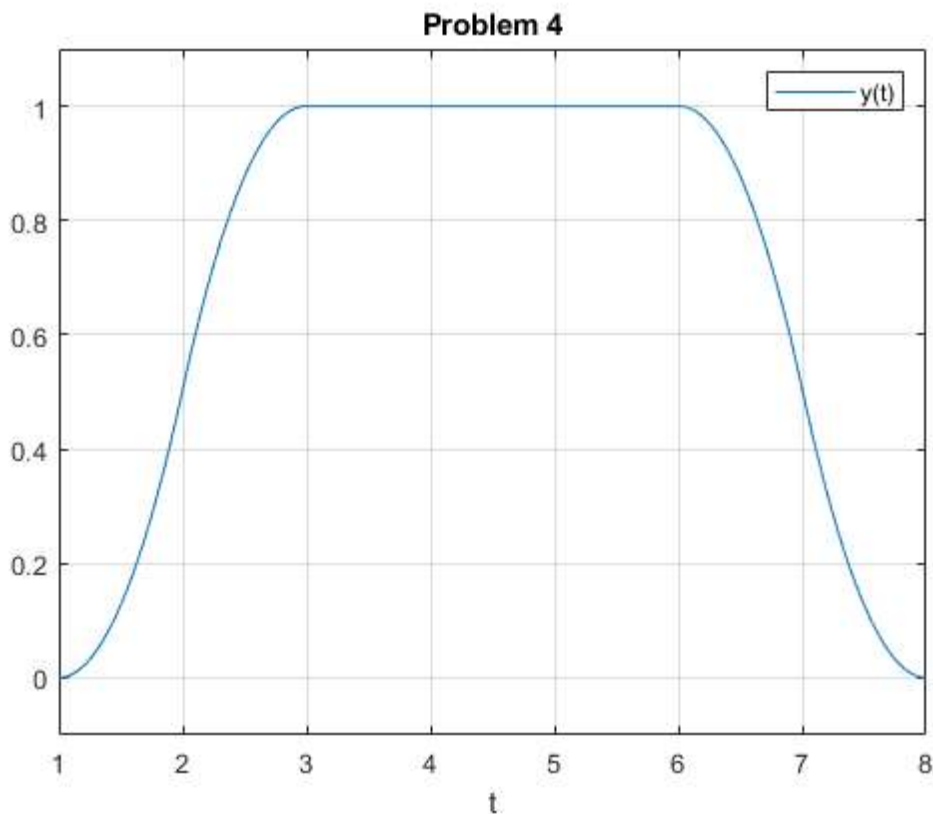


Problem 4

```
clear;
close all;
clc;

%{
x(t), as shown in the figure (see textbook) is given by  $x(t) = u(t+1) - u(t-4)$ , so  $x(t) = 1$  for  $t \in (-1, 4)$ , and  $x(t) = 0$  elsewhere.  $h(t)$  as shown is given by  $h(t) = (t-1)[u(t-1) - u(t-2)] + (-t+3)[u(t-2) - u(t-3)]$ ; that is,  $h(t) = t-1$  on  $t \in (1, 2)$  and  $h(t) = -t+3$  on  $t \in (2, 3)$ .
%}

%taking limits, a=-1, b=4, compute time interval
a=-1;b=4;
step=.01; t=a:step:b;
%define x(t) and h(t) on the above interval
x=ones(size(t));
t1=-1:step:1-step; t2=1:step:2-step; t3=2:step:3-step; t4=3:step:4;
h1=zeros(size(t1));h2=t2-1;h3=-t3+3;h4=zeros(size(t4));
h=[h1 h2 h3 h4];
plot(t,h);
%convolve x and h (note that conv() inherently performs discrete time
%convolution from sampled inputs, and does not have internal access to the
%step size of the input arrays.
y=conv(x,h)*step;
%create new time vector for output signal y(t);
ty=-1:step:9;
plot(ty,y);xlim([1,8]);ylim([min(y)-range(y)*0.1, max(y)+range(y)*0.1]);
xlabel('t'); legend('y(t)'); grid on; title('Problem 4');
```



Problem 6

```
clear;
close all;
clc;

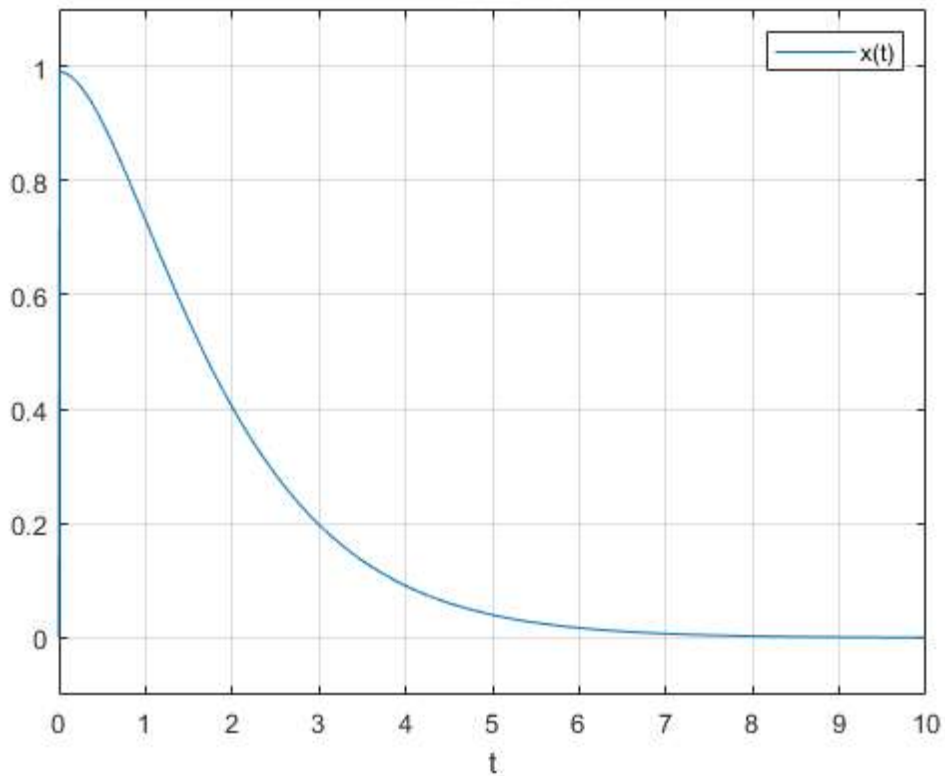
%{
Finding the input x(t) to a system based on the output y(t) and the impulse
response h(t) can be done using the deconv() function in MATLAB. Similarly
to conv(), we will need to normalize by the unit step, but in this case we
will need to divide y(t) by the step size, and ensure that its domain is
twice as large as h(t)'s.
%}

%define h(t) and y(t) on reasonable intervals
step=0.01;
t=0:step:10; yt=0:step:20;
h=exp(-2.*t); y=yt.*exp(-yt);

% deconvolve y and t (order matters, deconvolution is not commutative)
x=deconv(y,h)/step;

%plot against t
plot(t,x);xlim([0,10]);ylim([-0.1,1.1]);
xlabel('t');legend('x(t)');grid on;title('Problem 6');
```

Problem 6



Problem 9

```
clear;
close all;
clc;

%{
We are given two parallel connected systems S_1 and S_2, with separate
impulse responses. As shown in the textbook, the total output is the sum
of the responses of the two systems to the input signal  $x(t)$ , and so
 $h(t) = h_1(t) + h_2(t)$ . To find the total response, we can look at each
impulse response separately and sum the results, then integrate its
absolute value for all  $t$ . If the result is bounded (not infinite), we can
assume the system is BIBO-stable; otherwise, it is not.
%}

% First, we'll sum the two impulse responses to get the total impulse
% response  $h(t) = h_1(t) + h_2(t)$ :

syms h(t);
h = (exp(-3*t) + t*exp(-2*t))*heaviside(t);

% Then, check for stability by absolutely integrating the result
result = int(abs(h),t,-inf, inf);
% Check whether the result is bounded/unbounded
% isfinite() should return 0 if we get back inf, Inf, NaN, etc, but returns
% 1 even if the result is 0.
if isfinite(result)
    disp('The system is BIBO-stable');
else
```

```
    disp('The system is not BIBO-stable');  
end
```

The system is BIBO-stable

Published with MATLAB® R2024a